HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I

# BÁO CÁO ĐỀ CƯƠNG GIỮA KỲ

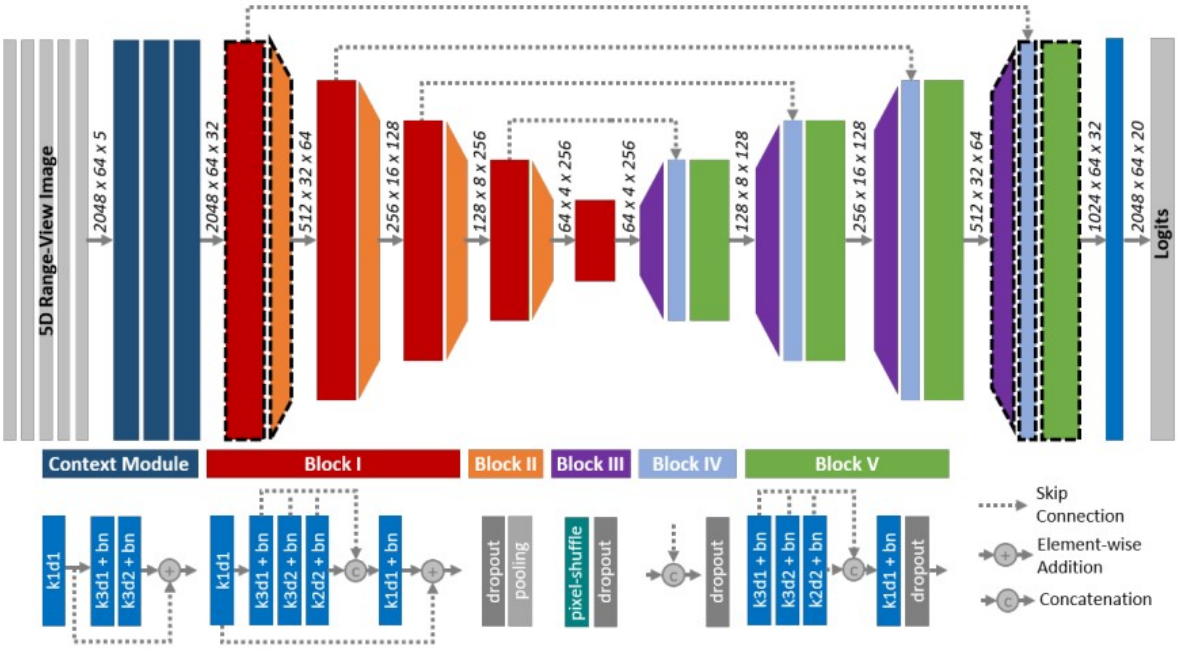
## HỌC PHẦN: THỰC TẬP CƠ SỞ

## ĐỀ TÀI

**Giảng viên hướng dẫn** : **TS. Kim Ngọc Bach**
**Mã sinh viên** : **Nguyễn Nhật Huy**
**Tên sinh viên** : **B23DCCN397**
**Lớp hành chính** : **D23CQCN05-B**
**Nhóm thực tập** : **11**

*Hà Nội – 2026*

I. Thực hiện cấu trúc SalsaNext và chạy demo cho 3D Semantic Segmantation



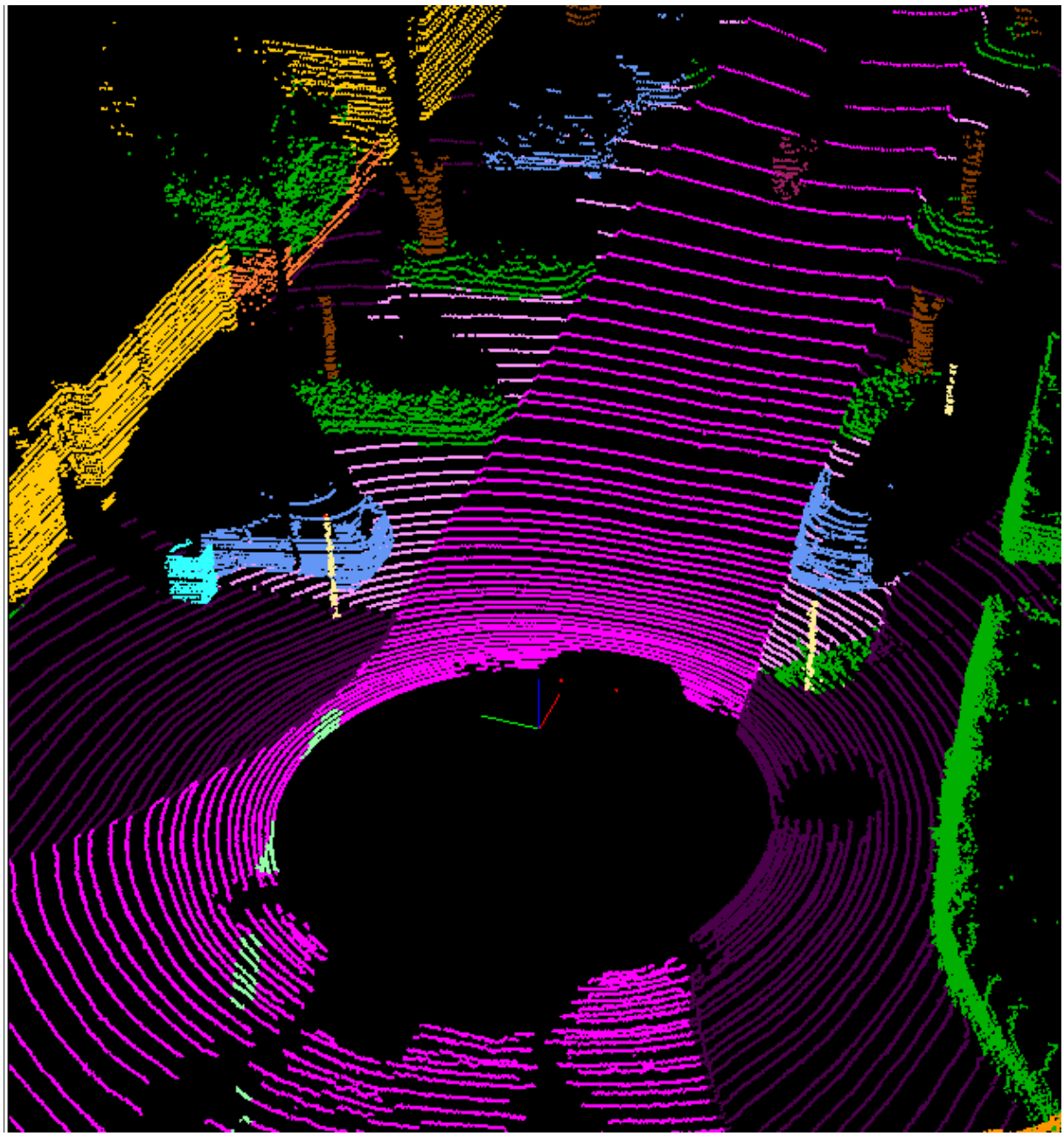

Hình 1: Cấu trúc SalsaNext

**Tuần này, em tập trung thực hiện việc implement kiến trúc của mô hình SalsaNext cho bài toán LiDAR semantic segmentation.**

**Các công việc đã thực hiện bao gồm:**

- Xây dựng các hàm đọc dữ liệu và nhãn từ file để phục vụ cho quá trình **training và validation**.
- Implement kiến trúc chính của mô hình SalsaNext theo thiết kế trong bài báo.
- Kiểm tra và phân tích một số hạn chế trong quá trình huấn luyện, từ đó đề xuất các hướng cải thiện như điều chỉnh **cấu trúc mạng** hoặc **hàm loss** nhằm nâng cao **accuracy**, đặc biệt đối với các **point cloud ở khoảng cách xa**.

1. Ban đầu, cần implement class để có thể đọc data phục vụ cho việc training và validation
2. Tiếp đến, cần viết hàm để chiếu data từ point cloud sang 2D image bằng projection based method



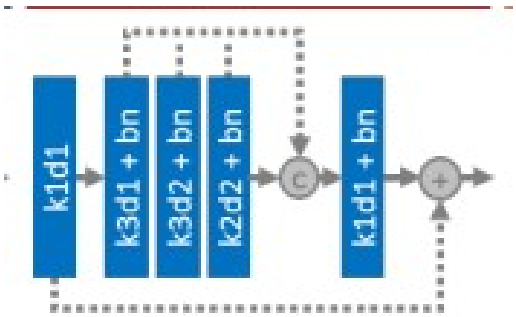
- Với mỗi point cloud, ta có dữ liệu đọc được có dạng (x, y, z, r)

Trong đó  x: Khoảng cách phía trước ( vector màu đỏ)
y : Khoảng cách hai bên(vector màu xanh lá)
z : Khoảng cách phía trên(vector màu xanh biển)

Ta sẽ chiếu vào một image có chiều cao H, chiều rộng W với công thức
- Với mỗi point cloud , ta chiếu lên điểm (u , v) trên image

$$\begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} \frac{1}{2}[1-\arctan(y,x)\Pi^{-1}]\omega \\ [1-(\arcsin(z,r^{-1})+f_{down})f^{-1}]h \end{pmatrix}$$

Trong đó:
h, w: chiều cao, chiều rộng image
$r=\sqrt{x^2+y^2+z^2}$
$f=|f_{down}|+|f_{up}|$
fdown , fup là hyperparameter
3. Viết cấu trúc SalsaNext
* Cấu trúc SalsaNext
- Contextual Module: Lớp màu xám đầu tiên ở hình 1. Nó giải quyết vấn đề của projection based method đó là làm mất đi ngữ cảnh (vị trí của các point cloud trong data). Để giải quyết vấn đề này, SalsaNext thêm các CNN với kernel size 1 x 1, 3 x 3 để học các ngữ cảnh, mối quan hệ giữa các point cloud
- Dilated Convolution: Lớp này để trích xuất các đặc trưng (features) - đây là lớp encoder. Mô hình sử dụng nhiều block để học các features sau đó sử dụng 1 CNN 1 x 1 kết hợp với mỗi output của mỗi lớp để học được nhiều đặc trưng hơn



```python
lass ResBlock(nn.Module):
   def __init__(self, in_filters, out_filters, dropout_rate, kernel_size=(3, 3), stride=1,
                pooling=True, drop_out=True):
       super(ResBlock, self).__init__()
       self.pooling = pooling
       self.drop_out = drop_out
       self.conv1 = nn.Conv2d(in_filters, out_filters, kernel_size=(1, 1), stride=stride)
       self.act1 = nn.LeakyReLU()

       self.conv2 = nn.Conv2d(in_filters, out_filters, kernel_size=(3,3), padding=1)
       self.act2 = nn.LeakyReLU()
       self.bn1 = nn.BatchNorm2d(out_filters)

       self.conv3 = nn.Conv2d(out_filters, out_filters, kernel_size=(3,3),dilation=2, padding=2)
       self.act3 = nn.LeakyReLU()
       self.bn2 = nn.BatchNorm2d(out_filters)

       self.conv4 = nn.Conv2d(out_filters, out_filters, kernel_size=(2, 2), dilation=2, padding=1
       self.act4 = nn.LeakyReLU()
       self.bn3 = nn.BatchNorm2d(out_filters)

       self.conv5 = nn.Conv2d(out_filters*3, out_filters, kernel_size=(1, 1))
       self.act5 = nn.LeakyReLU()
       self.bn4 = nn.BatchNorm2d(out_filters)

       if pooling:
           self.dropout = nn.Dropout2d(p=dropout_rate)
           self.pool = nn.AvgPool2d(kernel_size=kernel_size, stride=2, padding=1)
       else:
           self.dropout = nn.Dropout2d(p=dropout_rate)
```

codesnap.dev

- Kế tiếp lớp Dilated Convolution là lớp dropout, pooling giúp giảm kích thước, giảm overfitting giúp model học tốt hơn
- Pixel-shuffle Layer: Trái với cấu trúc SalsaNet dùng Inverse Convolution để upsampling, SalsaNext dùng Pixel - shuffle layer với ý tưởng xáo và chuyển pixel từ channel sang Height và Width

(H x W x Cr^2) => (Hr x Wr x C)

```python
upA = nn.PixelShuffle(2)(x)
```

- Về hàm loss, SalsaNext dùng hàm loss Lovasz-Softmax loss

4. Sau khi train, ta có thể thấy SalsaNext có độ chính xác mIoU = 59.5 (chấp nhận được)

* Mục tiêu tuần tiếp theo: Cố gắng cải tiến model, hàm loss để mIoU đạt 65%, tìm hiểu về phương pháp voxel based method để xử lý data sparse ( chiếu point cloud sang các voxel) và 3D CNN